

Policy Gradient Fine-Tuning of Diffusion Models with Timestep-Aware Optimization

AMANDA JORGE MENDES

ADVISOR: RODRIGO ANDRADE DE BEM



Agenda



- 01 **Introduction:** Why and how do we fine-tune diffusion models?
- 02 **Methodology:** Reframing Diffusion Model Fine-Tuning as a Sequential Decision Problem
- 03 **Preliminary Results:** Empirical Evaluation of Timestep Sampling and Reward Optimization
- 04 **Conclusion:** Conclusions and Next Steps

INTRODUCTION

—

Why and **how** do we fine-tune diffusion models?



Diffusion Models in Generative AI

- ✓ Generative framework for complex, high-dimensional data
- ✓ Achieve state-of-the-art results in image synthesis

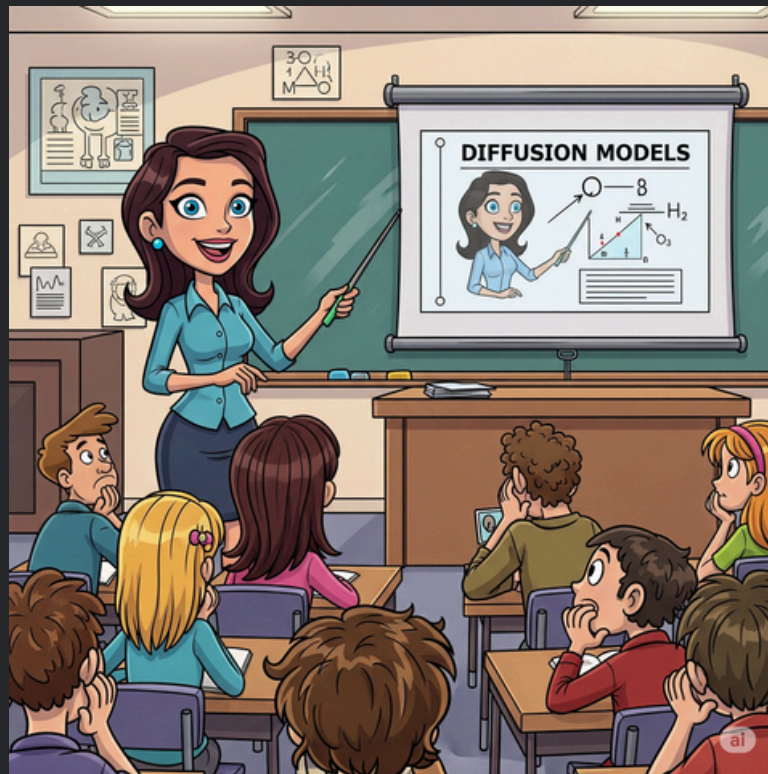


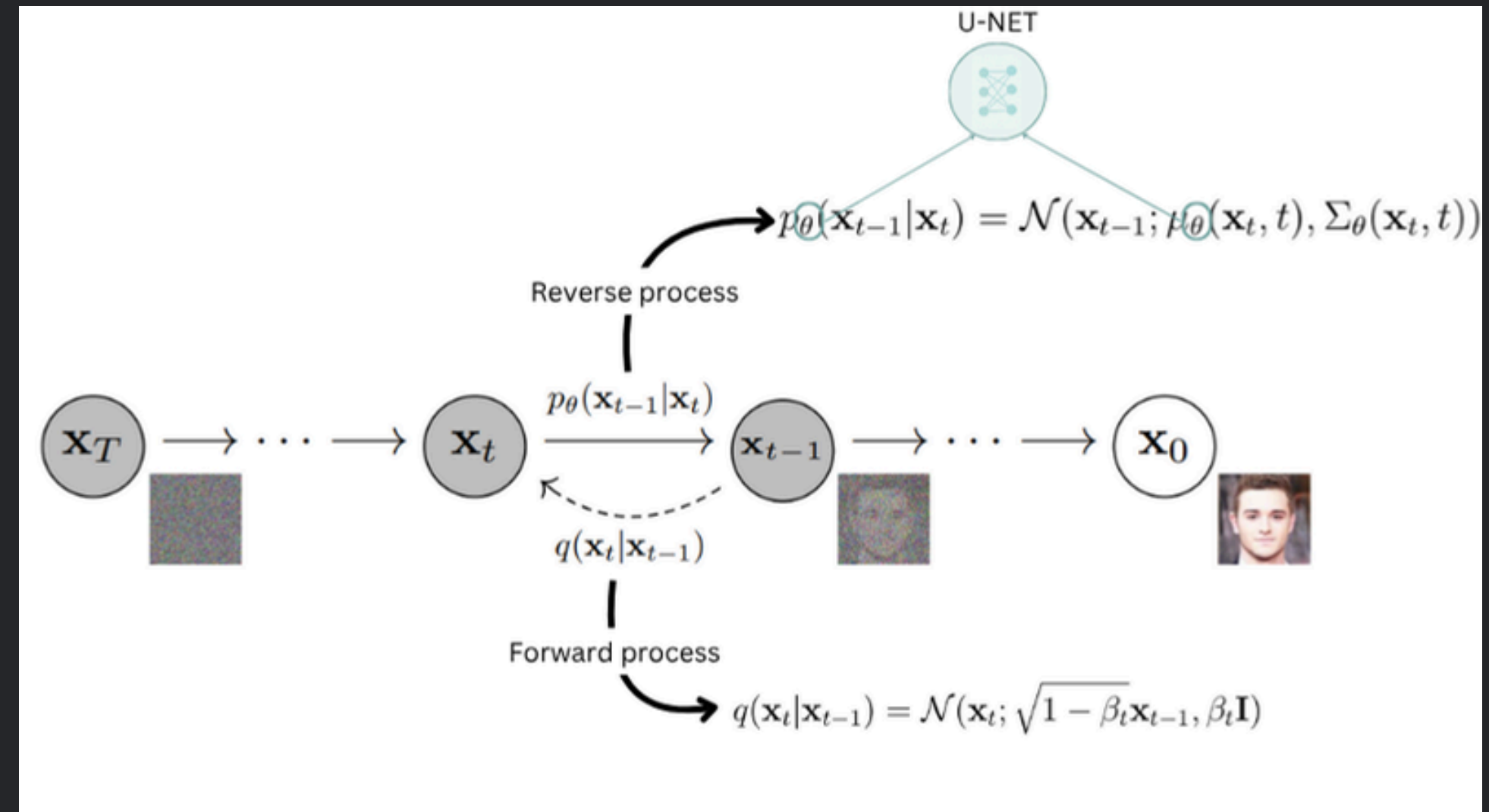
Imagen (Google DeepMind): “A cartoon-like image of a classroom with a woman presenting a slides presentation about diffusion models.”



Stable Diffusion (Stability AI): “Create an ultra-realistic professional photo shoot with soft lighting of an erupting volcano.”

Diffusion Models in Generative AI

- ✓ **Forward process:** gradually add noise to data
- ✓ **Reverse process:** denoise step by step
- ✓ Train diffusion model = learn reverse (denoising) process
- ✓ Each timestep corresponds to a specific level of noise



Misalignment with Downstream Objectives

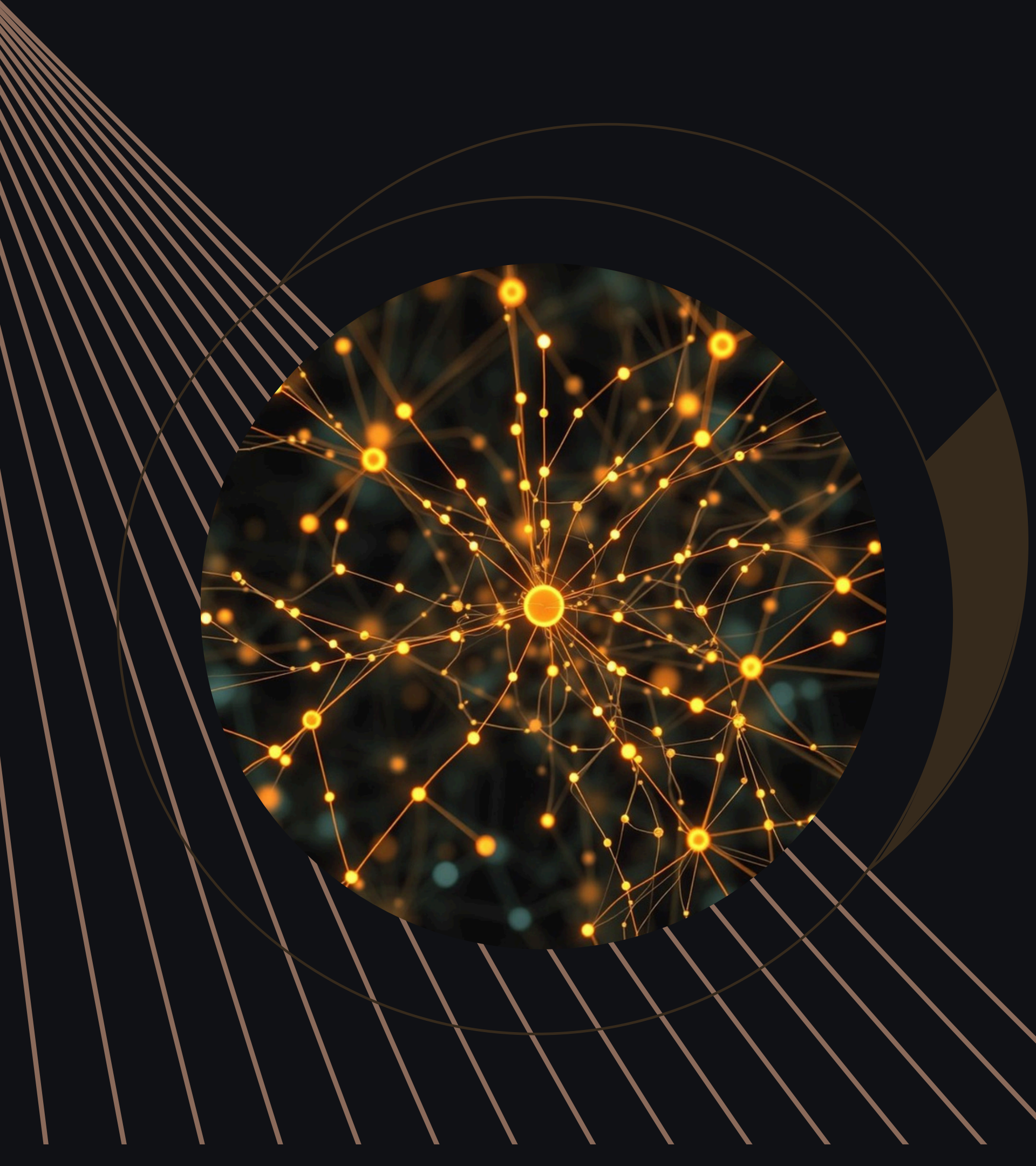
- Diffusion models are pretrained as foundation models on large, uncurated web-scale datasets
- Optimized using maximum likelihood
 - Promotes fidelity to the training distribution
 - Ignores task-specific or user-defined goals
- Outputs can reflect undesirable biases and fail to align with downstream requirements



Dall-E: "A green racoon washing dishes."



Stable Diffusion: "A team of engineers developing and testing an autonomous vehicle."



How to Align Foundation Models for Specific Tasks?

FINE-TUNING!



Fine-Tuning Diffusion Models

Supervised Fine-Tuning

Using Auxiliary Inputs

Reward-Conditioned generation

Prompt Encoder Tuning with Reward Feedback

Refinement of Intermediate Representations

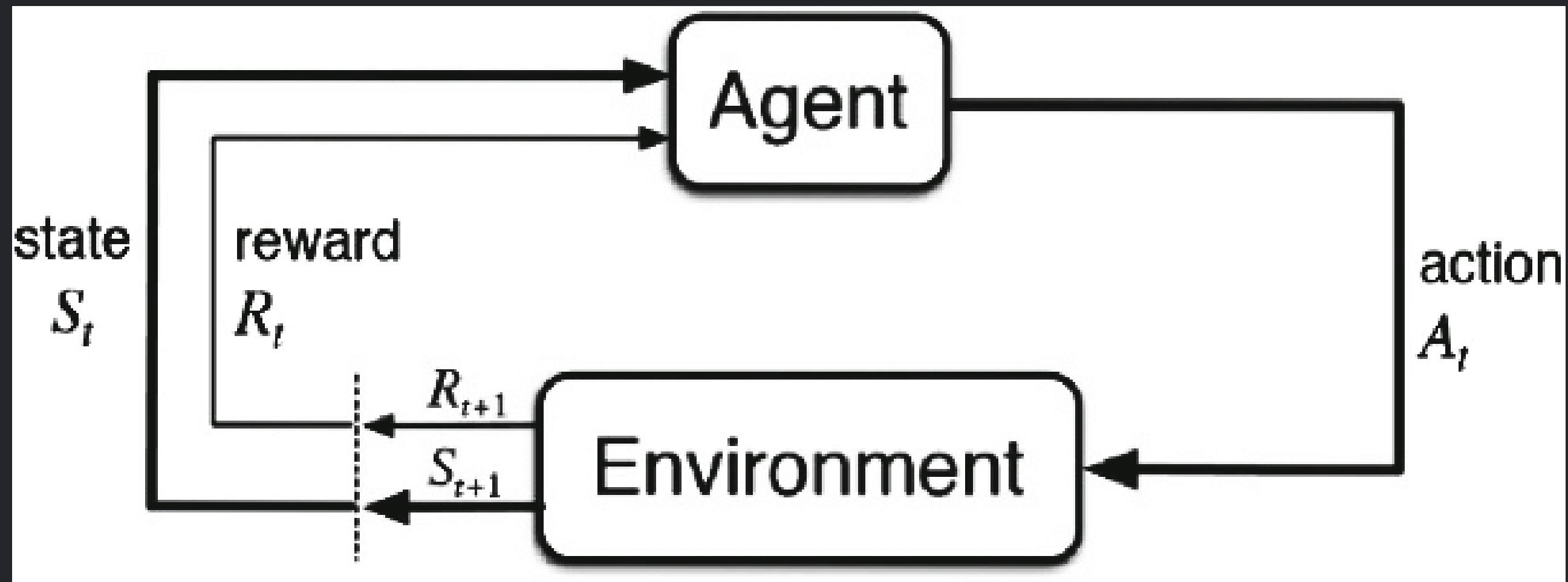
Reinforcement Learning: Policy Gradient Approaches

Differentiable reward optimization

Preference-based Fine-Tuning

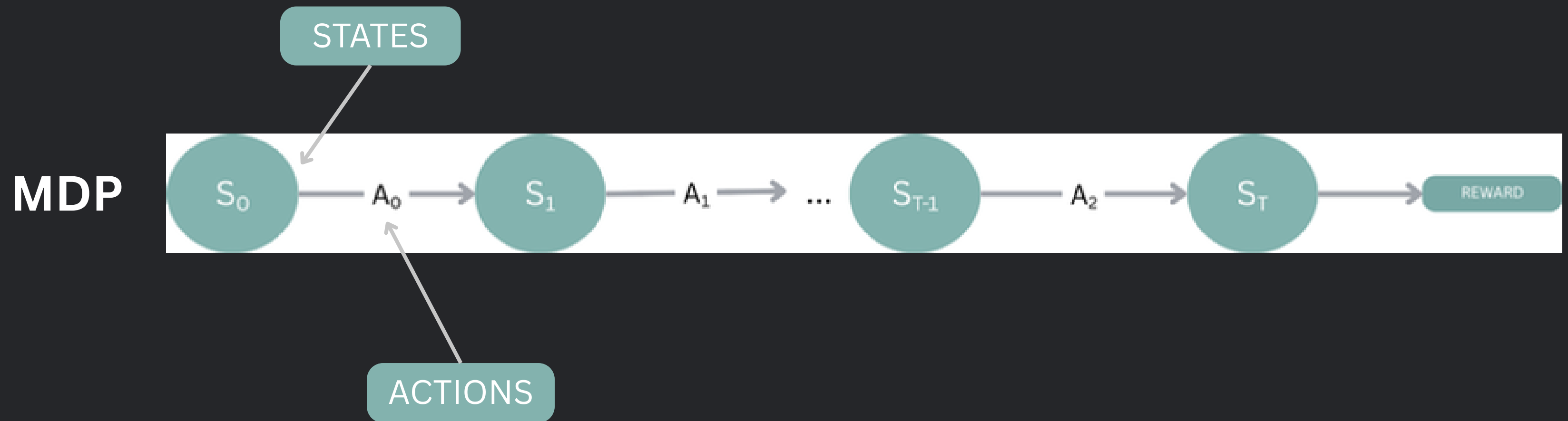
Reinforcement Learning

- A learning paradigm in which agents interact with an environment to achieve goals
- The objective is to learn a policy (π) that maximizes expected cumulative reward
- Mathematically formulated as a Markov Decision Process (MDP) —a model for stochastic decision-making



Reinforcement Learning

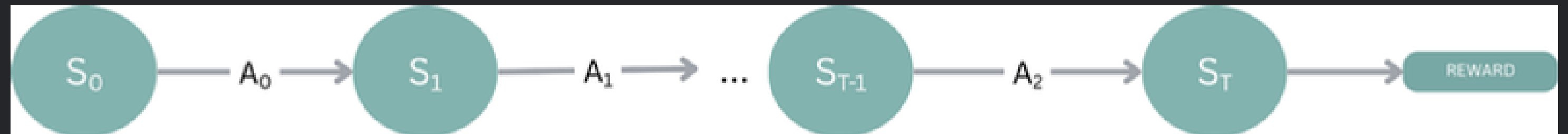
- A learning paradigm in which agents interact with an environment to achieve goals
- The objective is to learn a policy (π) that maximizes expected cumulative reward
- Mathematically formulated as a Markov Decision Process (MDP) —a model for stochastic decision-making



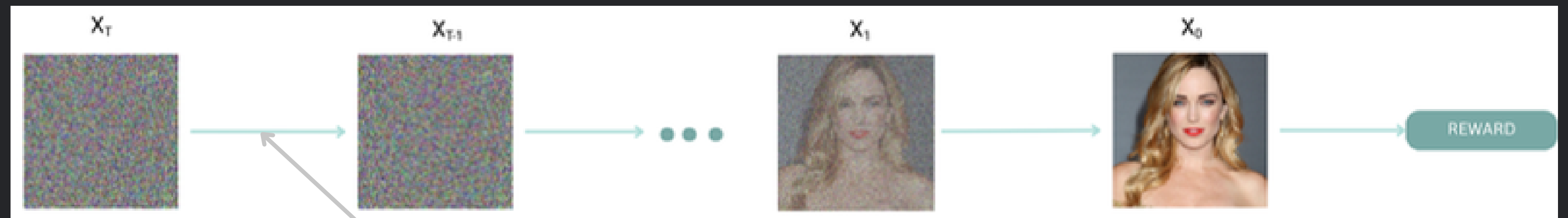
Fine-Tuning Diffusion Models with RL

To fine-tune a diffusion model with RL, we frame the diffusion process as an MDP, where each timestep is a state and the predicted denoising step is an action.

MDP



DIFFUSION



DENOISING STEP

Advantages of RL Methods in Diffusion

1 No Need for Labeled Data

Learns from scalar reward signals,
removing the need for curated datasets

2 Optimizes Task-Specific Objectives

Directly targets metrics that matter for
the downstream task

3 Handles Non- Differentiable Rewards

Can optimize goals that aren't
expressible as losses

4 Extends Beyond Training Distribution

Improves model behavior even in
underrepresented or misaligned regions
of pretraining data

Limitations of RL Methods in Diffusion

1 On-Policy Sampling

Requires extensive new rollouts after every single policy update.

2 Long Trajectories

Hundreds of denoising steps result in high computational costs.

3 High Variance

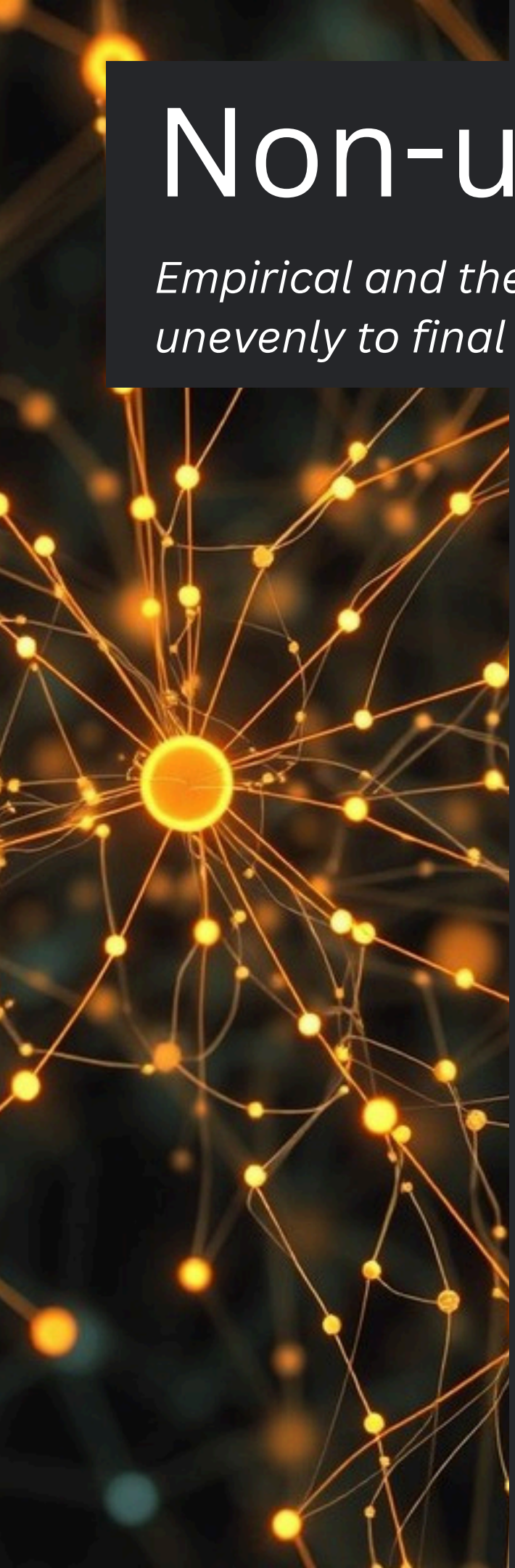
Monte Carlo gradient estimates are inherently noisy, leading to unstable learning.

4 Delayed Rewards

Feedback is only available at the very end of the generation process.

5 Inefficient Credit Assignment

Many actions might not significantly influence the final reward.



Non-uniform contribution of timesteps

Empirical and theoretical evidence show that timesteps in the diffusion process contribute unevenly to final output quality and learning dynamics during pre-training.

Different Loss
Magnitudes

Varying Signal-to-
Noise Ratio (SNR)

Perceptual
Differences

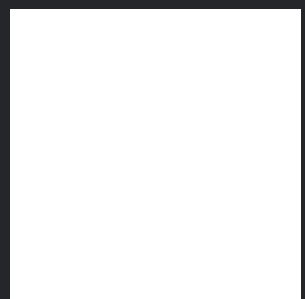
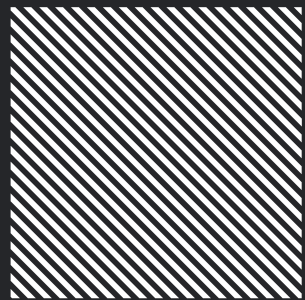
Gradient Variance

Wasserstein Distance

Gradient Conflicts

Hypothesis:

Timesteps likely contribute unevenly to the reward and, consequently, to the RL fine-tuning dynamics.



OBJECTIVES AND MAIN CONTRIBUTIONS

Objective: Improve the stability and efficiency of RL fine-tuning, achieving better results with less computation

Approach: Model denoising as a MDP in a RL framework and smartly sampling specific actions/timesteps

MAIN EXPECTED CONTRIBUTIONS:

- Systematic analysis of how individual timesteps affect convergence and reward in policy gradient fine-tuning
- Introduction and evaluation of timestep selection strategies: truncated uniform, Beta-based, and learnable distributions targeting informative diffusion regions.
- Empirical evidence that non-uniform timestep sampling improves sample efficiency and reward alignment.

METHODOLOGY

Reframing Diffusion Model Fine-Tuning as a Sequential Decision Problem



Policy Gradient

Intuition:

If an action led to a high return, increase the probability of choosing it again in similar states.

Key Idea

Directly optimizes a stochastic parameterized policy using gradient ascent

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)}$$

Objective

Maximize expected return

How?

Using Monte Carlo to estimate gradients

Policy Gradient

Intuition:

If an action led to a high return, increase the probability of choosing it again in similar states.

Key Idea

Directly optimizes a stochastic parameterized policy using gradient ascent

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)}$$

Objective

Maximize expected return

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [G_0]$$

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

How?

Using Monte Carlo to estimate gradients

Policy Gradient

Intuition:

If an action led to a high return, increase the probability of choosing it again in similar states.

Key Idea

Directly optimizes a stochastic parameterized policy using gradient ascent

$$\theta_{t+1} = \theta_t + \alpha \widehat{\nabla J(\theta_t)}$$

Objective

Maximize expected return

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}} [G_0]$$

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k$$

How?

Using Monte Carlo to estimate gradients → Unbiased but high variance

$$\nabla_{\theta} J(\theta) \approx \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t \right]$$

Proximal Policy Optimization (PPO)

- ✓ **Stable and efficient learning:** Improves over basic policy gradient
- ✓ **Surrogate objective:** Uses a clipped objective to limit large, unstable policy updates.
- ✓ **Advantage estimation:** Reduces gradient variance by estimating and normalizing how much better an action is than expected.
- ✓ **Importance sampling:** Corrects for the difference between the current and old policy when reusing data.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

Proximal Policy Optimization (PPO)

- ✓ **Stable and efficient learning:** Improves over basic policy gradient
- ✓ **Surrogate objective:** Uses a clipped objective to limit large, unstable policy updates.
- ✓ **Advantage estimation:** Reduces gradient variance by estimating and normalizing how much better an action is than expected.
- ✓ **Importance sampling:** Corrects for the difference between the current and old policy when reusing data.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

$$A(x_0, c) = \frac{r(x_0, c) - \mu}{\sigma}$$

Proximal Policy Optimization (PPO)

- ✓ **Stable and efficient learning:** Improves over basic policy gradient
- ✓ **Surrogate objective:** Uses a clipped objective to limit large, unstable policy updates.
- ✓ **Advantage estimation:** Reduces gradient variance by estimating and normalizing how much better an action is than expected.
- ✓ **Importance sampling:** Corrects for the difference between the current and old policy when reusing data.

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

$$A(x_0, c) = \frac{r(x_0, c) - \mu}{\sigma}$$

$$r_t(\theta) = \frac{\pi_{\theta}(x_{t-1} \mid x_t, c)}{\pi_{\theta_{\text{old}}}(x_{t-1} \mid x_t, c)}$$

From Policy Gradient to Diffusion

How do we apply policy gradient to diffusion?

Classic Policy Gradient

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Applying Policy Gradient to Diffusion

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau | c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

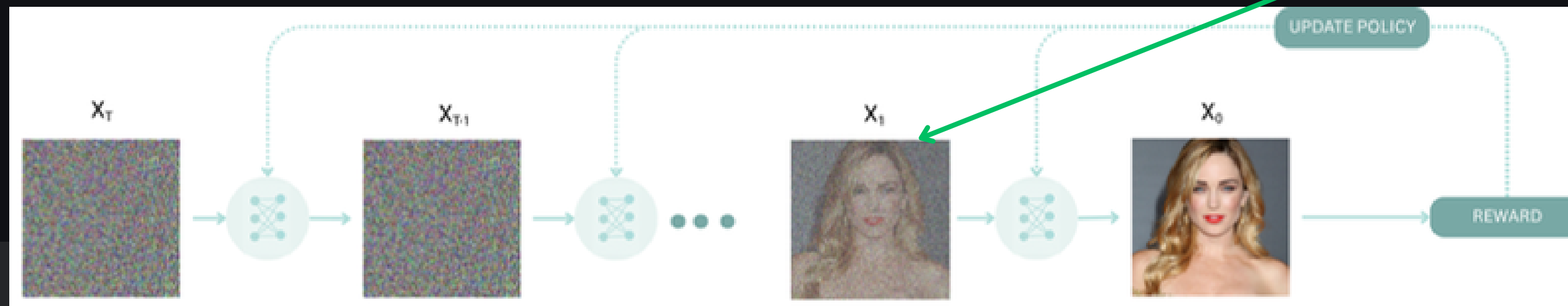
From Policy Gradient to Diffusion

Classic Policy Gradient

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t \mid s_t) \right]$$

Applying Policy Gradient to Diffusion

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} \mid x_t, c) \right]$$



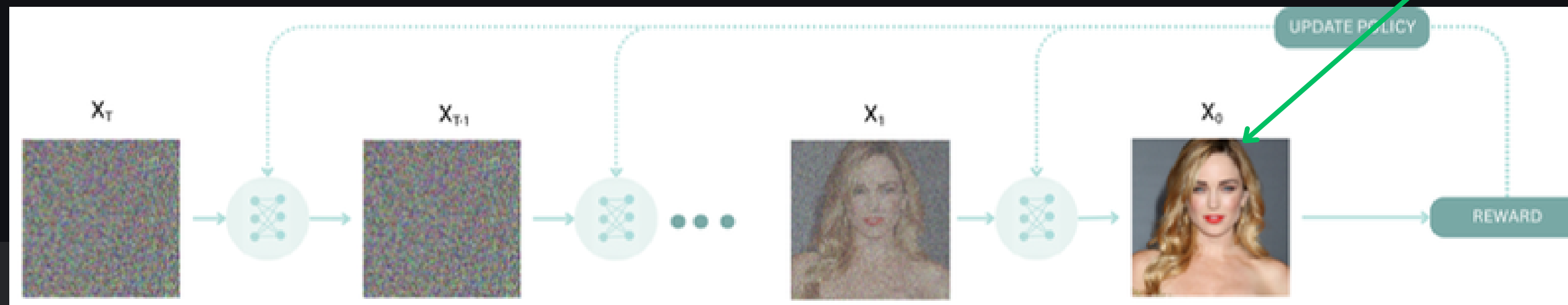
From Policy Gradient to Diffusion

Classic Policy Gradient

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Applying Policy Gradient to Diffusion

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau | c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$



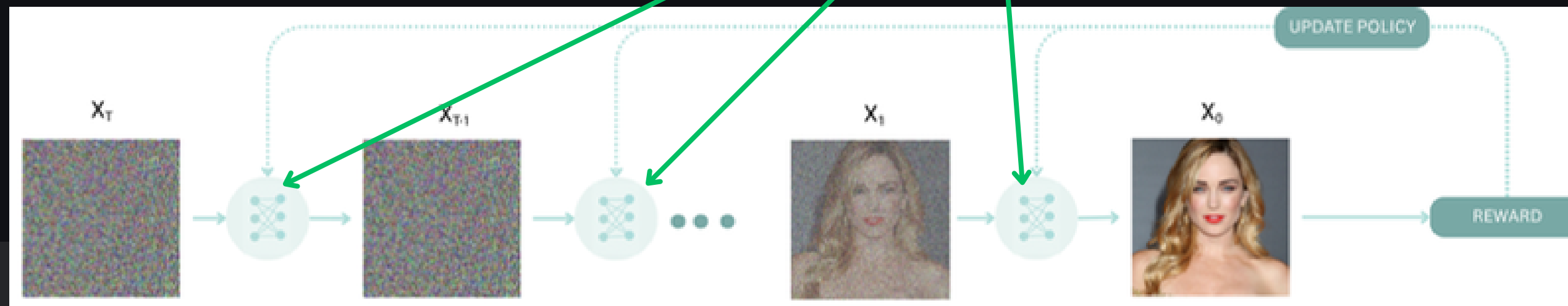
From Policy Gradient to Diffusion

Classic Policy Gradient

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} \left[\sum_{t=0}^T G_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Applying Policy Gradient to Diffusion

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau | c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$



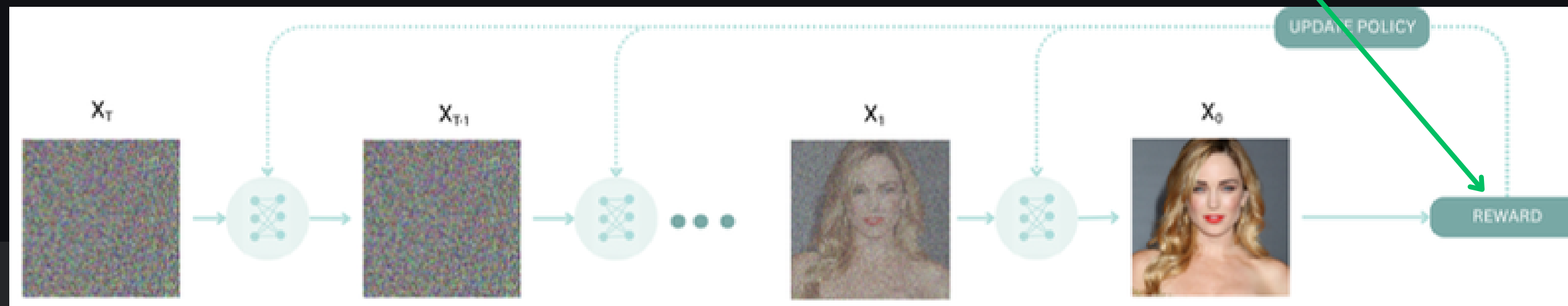
From Policy Gradient to Diffusion

Classic Policy Gradient

$$\nabla_{\theta} J(\theta) = E_{\pi_{\theta}} \left[\sum_{t=0}^T \underbrace{G_t}_{\text{green circle}} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right]$$

Applying Policy Gradient to Diffusion

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau | c)} \left[\underbrace{r(x_0, c)}_{\text{green circle}} \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$



Timestep Sampling

ORIGINAL

Naively computing the policy gradient requires summing over all T timesteps per update — expensive and inefficient.

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

TIMESTEP SAMPLING

The policy gradient can be reformulated as an expectation over timesteps

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c), t \sim h(t)} \left[\frac{1}{h(t)} r(x_0, c) \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

MONTE CARLO ESTIMATE

Estimate the gradient by sampling timesteps from a distribution $h(t)$, instead of summing over all T . MC is unbiased, but has with higher variance.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} | x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

Timestep Sampling

ORIGINAL

Naively computing the policy gradient requires summing over all T timesteps per update — expensive and inefficient.

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

TIMESTEP SAMPLING

The policy gradient can be reformulated as an expectation over timesteps

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c), t \sim h(t)} \left[\frac{1}{h(t)} r(x_0, c) \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

MONTE CARLO ESTIMATE

Estimate the gradient by sampling timesteps from a distribution $h(t)$, instead of summing over all T . MC is unbiased, but has with higher variance.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} | x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

Timestep Sampling

ORIGINAL

Naively computing the policy gradient requires summing over all T timesteps per update — expensive and inefficient.

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

TIMESTEP SAMPLING

The policy gradient can be reformulated as an expectation over timesteps

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c), t \sim h(t)} \left[\frac{1}{h(t)} r(x_0, c) \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

MONTE CARLO ESTIMATE

Estimate the gradient by sampling timesteps from a distribution $h(t)$, instead of summing over all T . MC is unbiased, but has with higher variance.

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} | x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

Timestep Sampling

ORIGINAL

Naively computing the policy gradient requires summing over all T timesteps per update — expensive and inefficient.

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c)} \left[r(x_0, c) \sum_{t=1}^T \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

TIMESTEP SAMPLING

The policy gradient can be reformulated as an expectation over timesteps

$$\nabla_{\theta} J(\theta) = E_{c \sim p(c), \tau \sim p_{\theta}(\tau|c), t \sim h(t)} \left[\frac{1}{h(t)} r(x_0, c) \nabla_{\theta} \log p_{\theta}(x_{t-1} | x_t, c) \right]$$

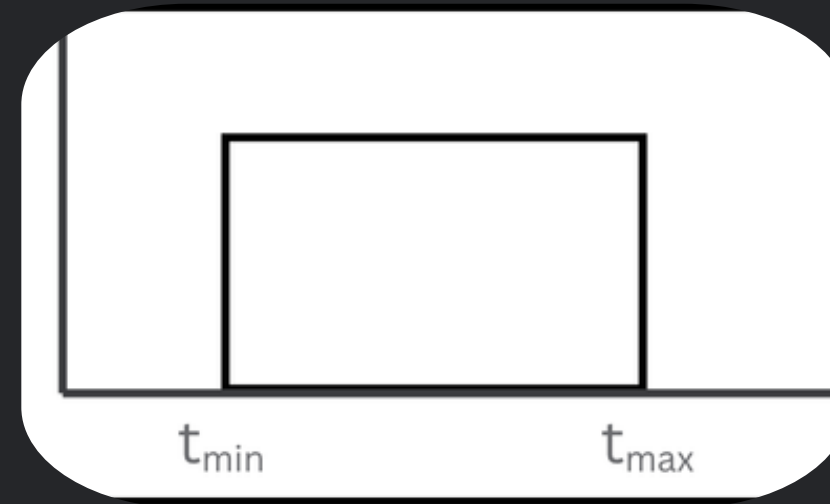
MONTE CARLO ESTIMATE

Estimate the gradient by sampling timesteps from a distribution $h(t)$, instead of summing over all T . MC is unbiased, but has with higher variance.

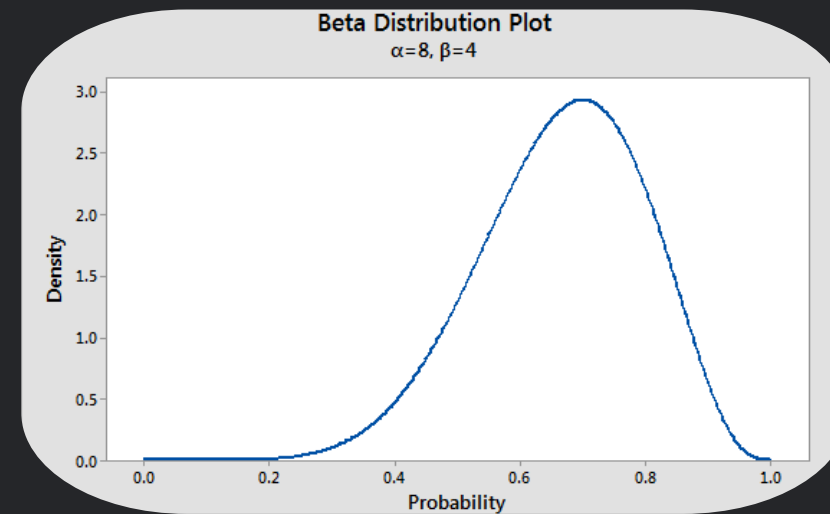
$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left(\frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} | x_{t_j^{(i)}}^{(i)}, c^{(i)}) \right)$$

Defining $h(t)$

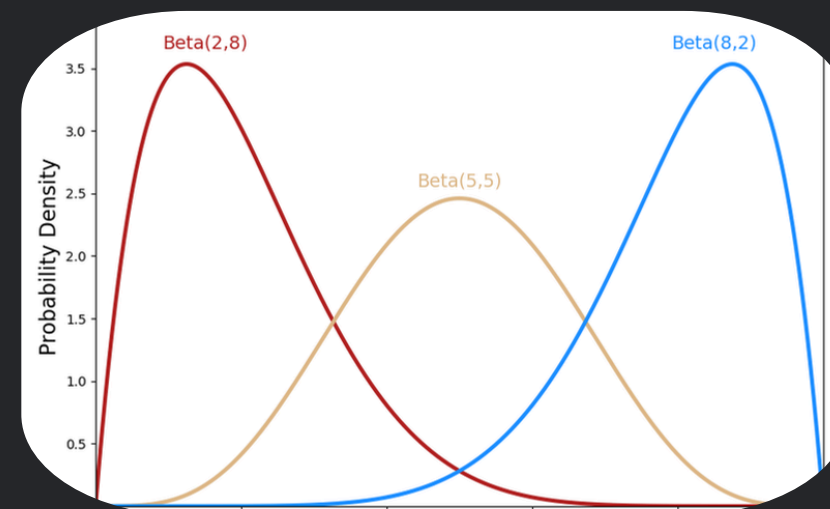
A well-designed $h(t)$ prioritizes the most relevant timesteps instead of treating them uniformly



TRUNCATED-UNIFORM



BETA DISTRIBUTION



LEARNABLE
DISTRIBUTION

PRELIMINARY RESULTS

—
Empirical Evaluation of Timestep Sampling and Reward Optimization



Experimental Setup



Pretrained Diffusion Model

Trained on CelebA-HQ images

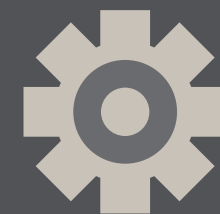


Image Generation

50 denoising steps



Generates ~78% female faces



Reward Function

$$r(x_0) = \text{IR}(x_0) + 2 \cdot \text{GenderReward}(x_0)$$

Visual Evolution Across Fine-Tuning

Early Steps:

Poor quality, sudden changes in content

Final Steps:

Clear images, subtle and consistent refinements

Gender shift

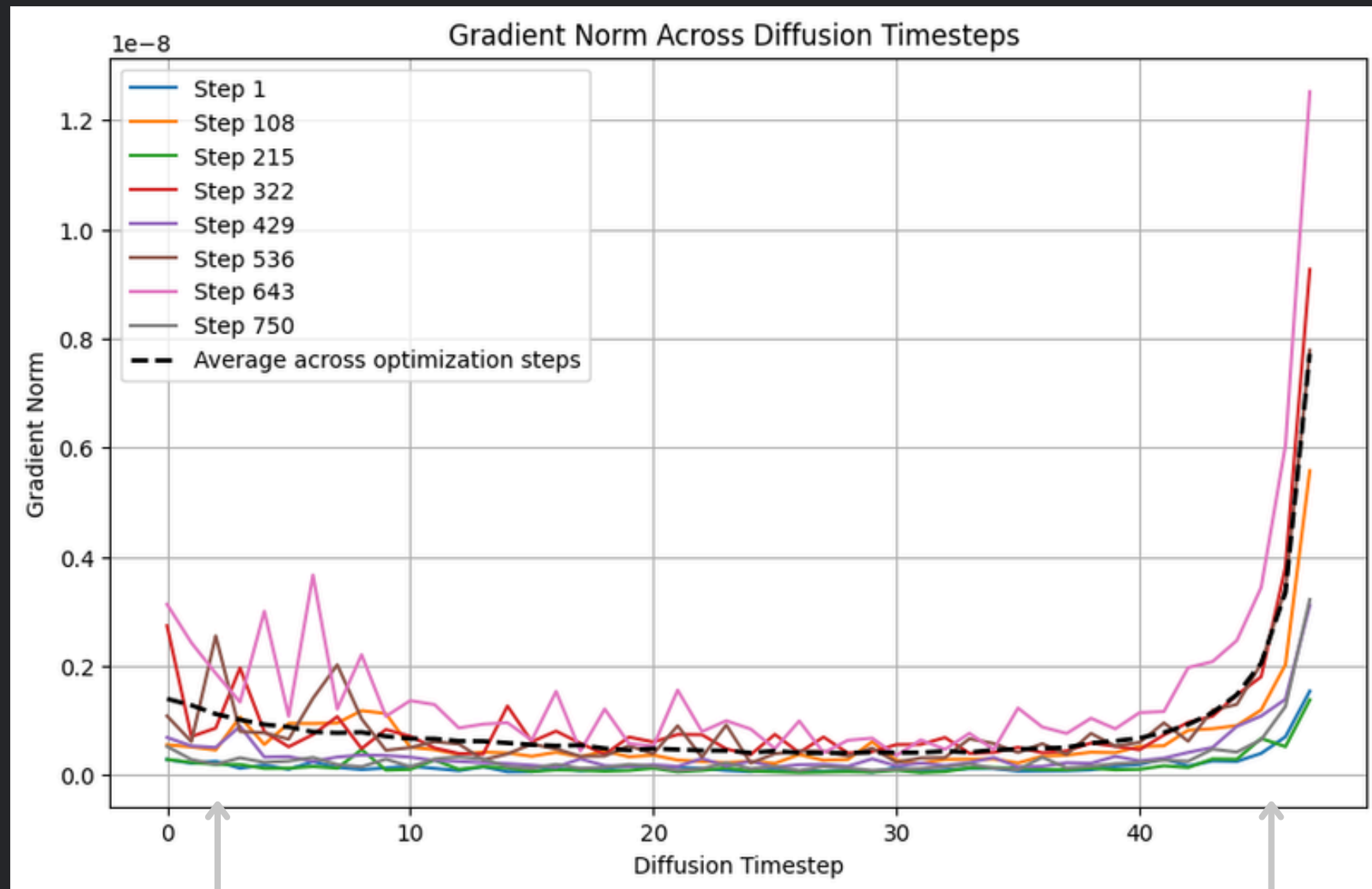
Increased male representation from 35% to 100%

Image quality

Transforms low-quality, distorted images into high-quality, refined, and stable outputs

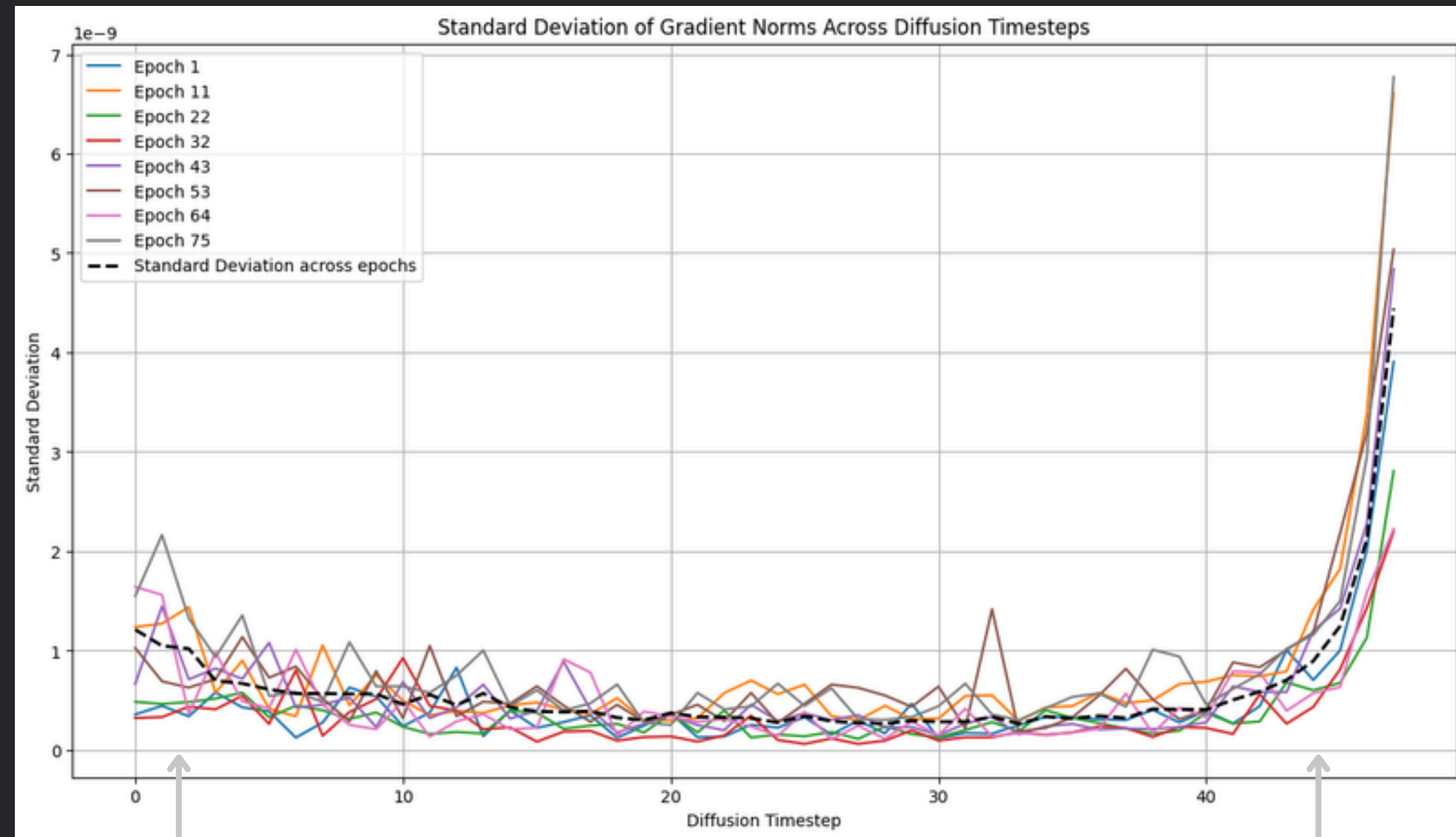


Gradient Across Timesteps



Pure noise

Clean image



Pure noise

Clean image

Truncated-Uniform Sampling

Analysing the effects of **truncating the timestep range** and **subsampling**

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

Truncated-Uniform Sampling

Analysing the effects of **truncating the timestep range** and **subsampling**

$$h(t) = \mathcal{U}(t_{\min}, t_{\max})$$

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

Truncated-Uniform Sampling

Analysing the effects of **truncating the timestep range** and **subsampling**

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} \mid x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

$$h(t) = \mathcal{U}(t_{\min}, t_{\max})$$

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

Truncated-Uniform Sampling

Analysing the effects of **truncating the timestep range** and **subsampling**

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} \mid x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

$$h(t) = \mathcal{U}(t_{\min}, t_{\max})$$

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

Truncated-Uniform Sampling

Analysing the effects of **truncating the timestep range** and **subsampling**

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} \mid x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

$$h(t) = \mathcal{U}(t_{\min}, t_{\max})$$

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

Truncated-Uniform Sampling

Analysing the effects of **truncating the timestep range** and **subsampling**

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} \mid x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

$$h(t) = \mathcal{U}(t_{\min}, t_{\max})$$

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

Truncated-Uniform Sampling

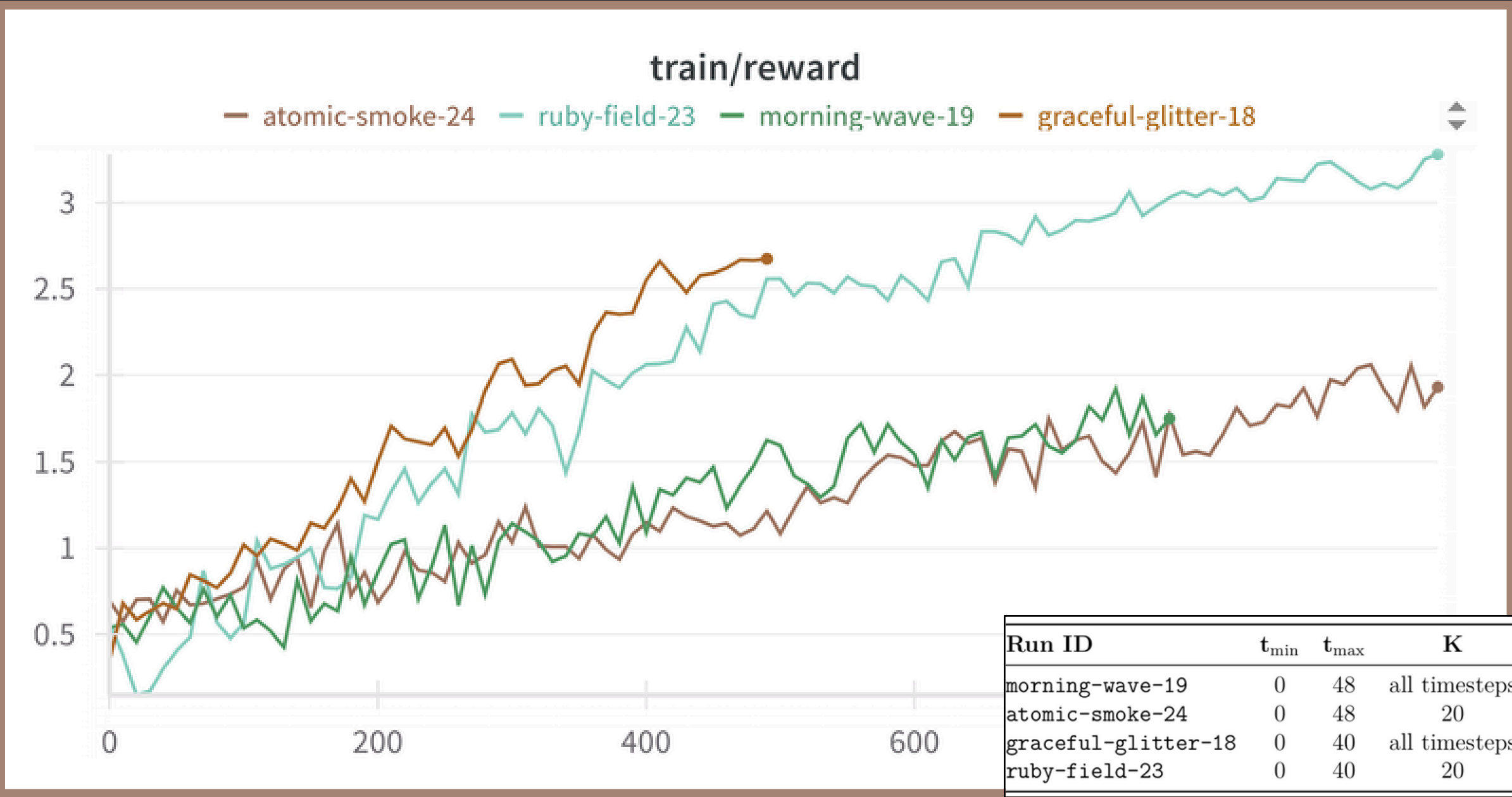
Analysing the effects of **truncating the timestep range** and **subsampling**

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \frac{1}{K} \sum_{j=1}^K \frac{1}{h(t_j^{(i)})} r(x_0^{(i)}, c^{(i)}) \nabla_{\theta} \log p_{\theta}(x_{t_j^{(i)}-1}^{(i)} \mid x_{t_j^{(i)}}^{(i)}, c^{(i)})$$

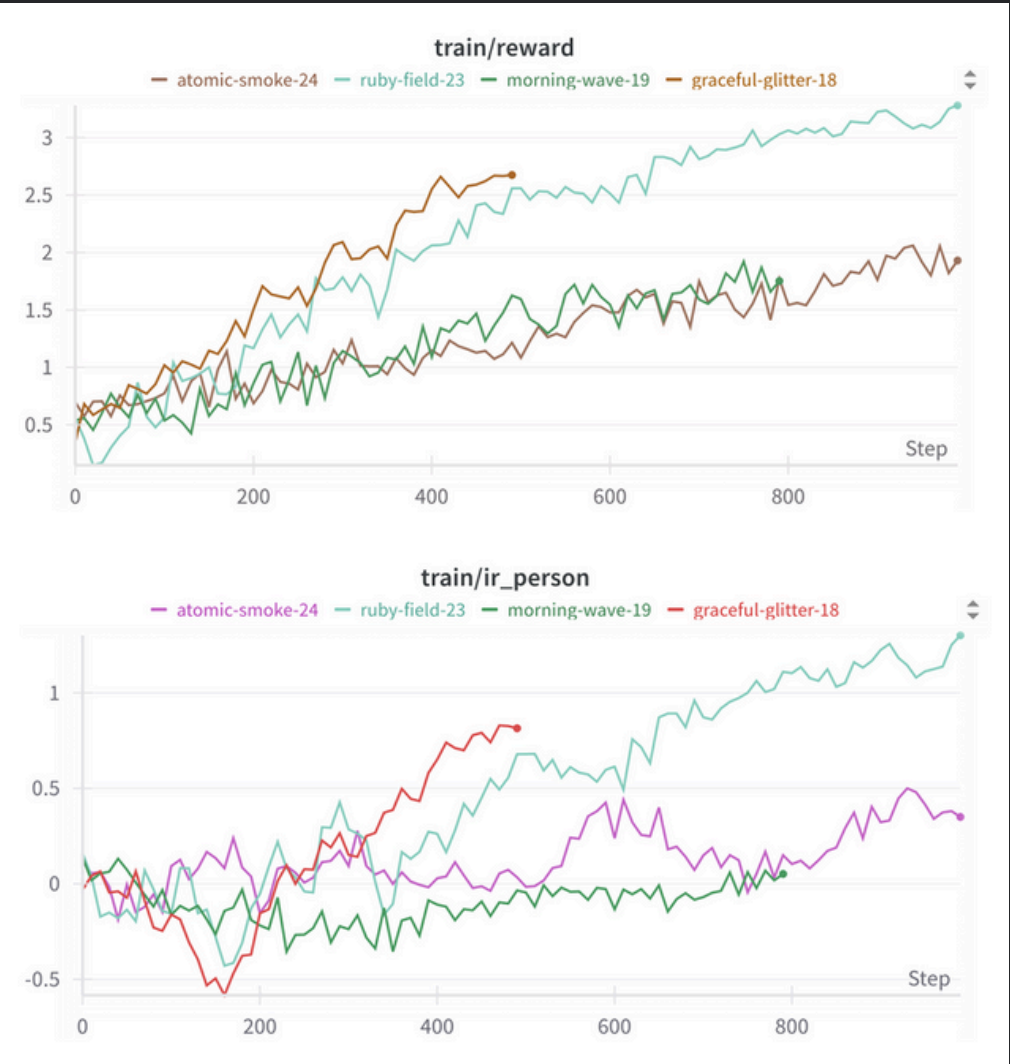
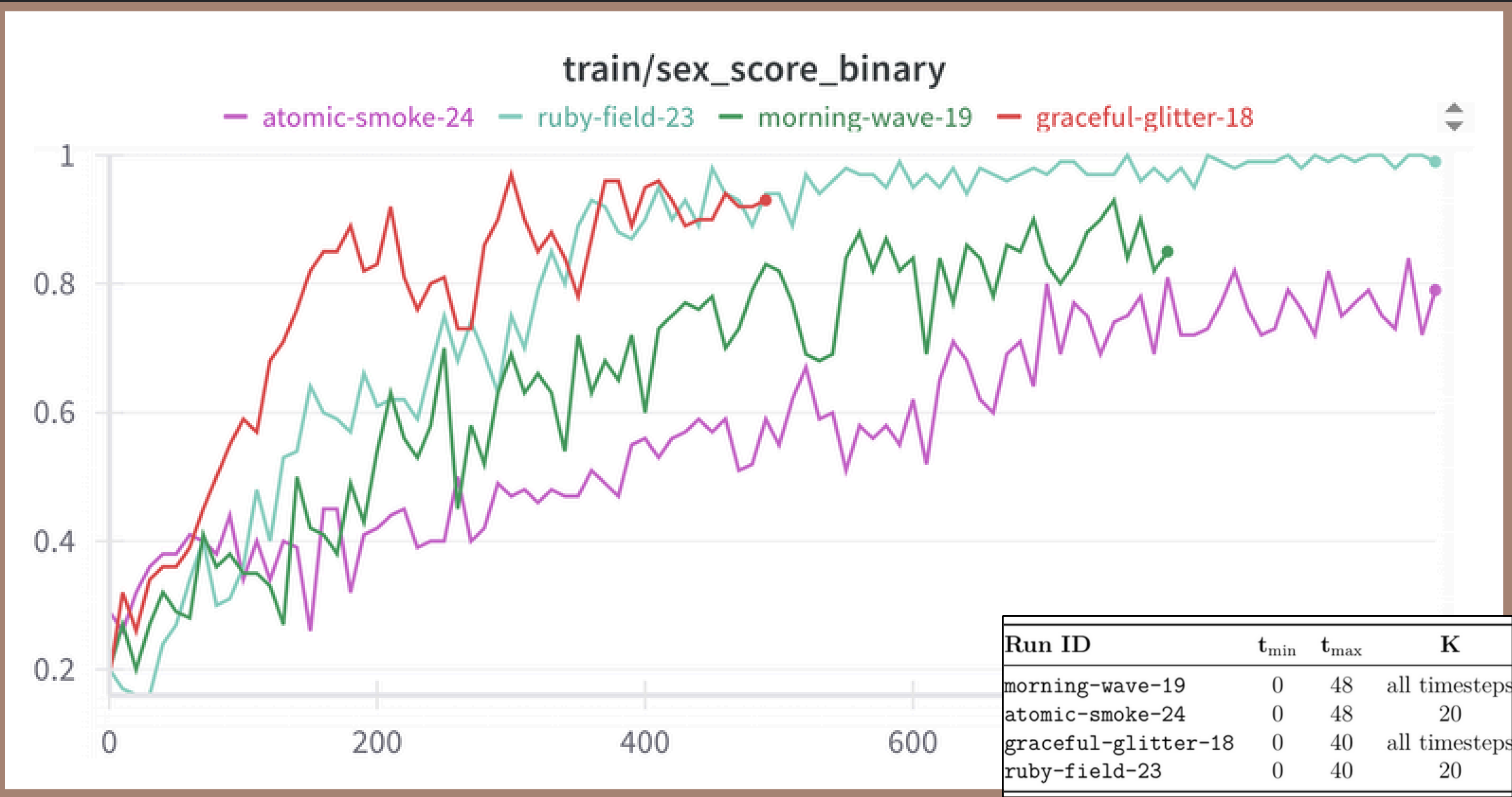
$$h(t) = \mathcal{U}(t_{\min}, t_{\max})$$

Run ID	$t_{\min}$	$t_{\max}$	K
morning-wave-19	0	48	all timesteps
atomic-smoke-24	0	48	20
graceful-glitter-18	0	40	all timesteps
ruby-field-23	0	40	20

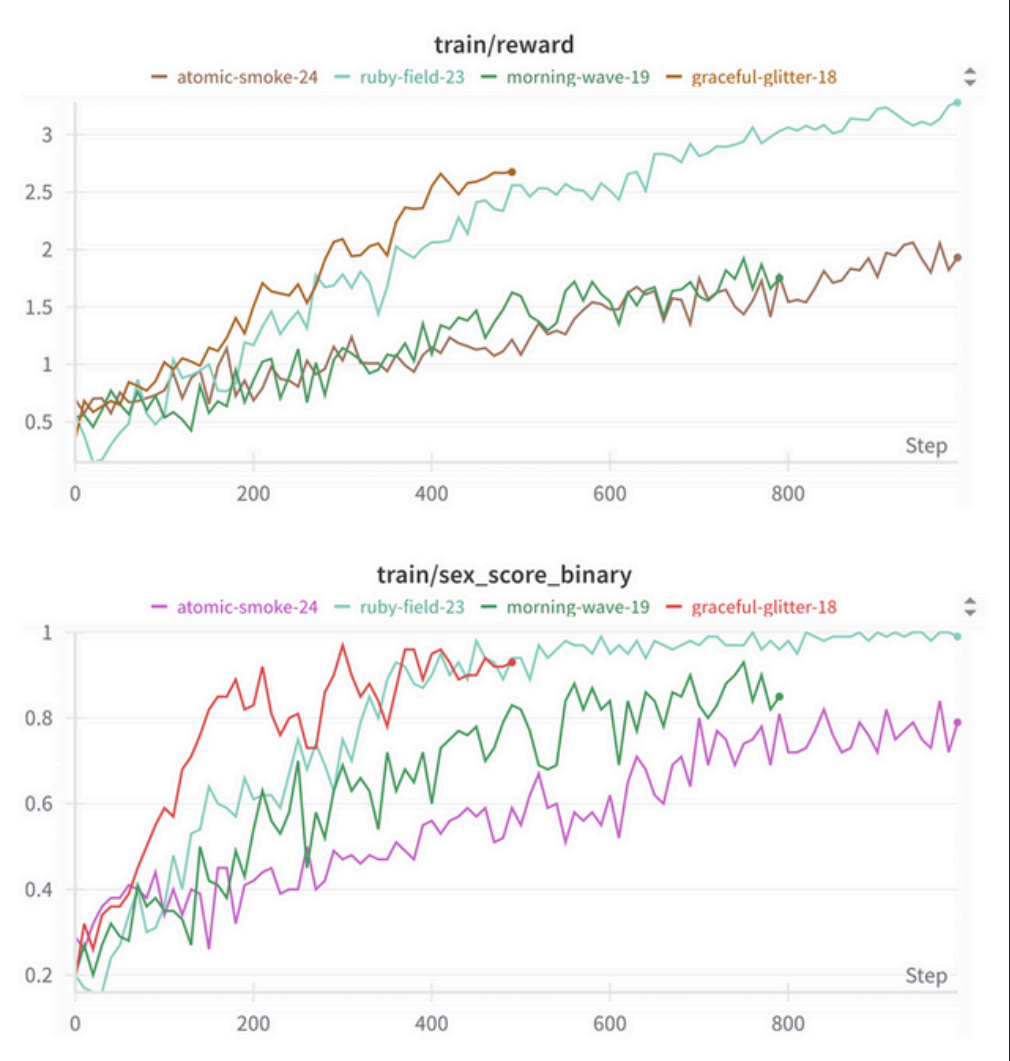
Fine-Tuning Dynamics



Fine-Tuning Dynamics



Fine-Tuning Dynamics



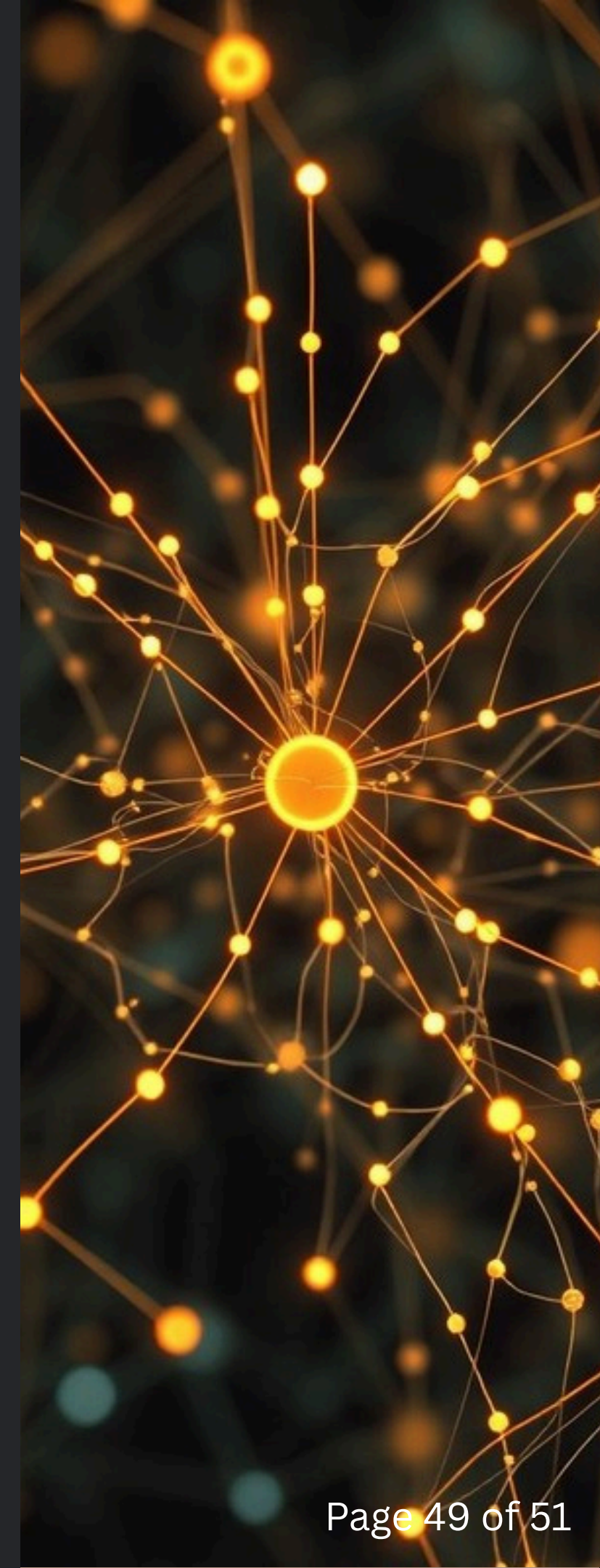
CONCLUSIONS

Policy Gradient Fine-Tuning of Diffusion Models with
Timestep-Aware Optimization: **Conclusions** and **Next Steps**



Conclusion: Timestep-Aware RL Fine-Tuning for Diffusion Models

- Policy gradient fine-tuning effectively shifted the output distribution of a pretrained diffusion model—improving image quality (via human preference rewards) and increasing the proportion of male faces
- Diffusion timesteps contribute unequally
- Sampling a subset of timesteps maintains or improves performance while reducing compute proportionally
- Excluding late, noisy steps consistently improves reward and sample efficiency
- Timestep-aware sampling is a key strategy for efficient reward-driven diffusion fine-tuning





NEXT STEPS

Timestep impact analysis

Extend the gradient-based analysis, inspired by timestep-aware supervised training.

Support size sensitivity

Investigate how the number of sampled steps (K) affects reward, efficiency, and gradient variance.

Expanded $h(t)$ exploration

Test broader timestep distributions, including Beta, dynamic, and extended truncated variants.

Reward-specific timestep influence

Assess whether different rewards benefit from focusing on different timestep regions.

Baseline comparison

Benchmark against existing fine-tuning methods to validate effectiveness and efficiency gains.

Toward a general methodology

Define a robust, generic procedure for policy gradient fine-tuning with timestep sampling.

Bibliography

- HO, J.; JAIN, A.; ABBEEL, P. Denoising diffusion probabilistic models. Advances in neural information processing systems, v. 33, p. 6840–6851, 2020.
- SUTTON, R.; BARTO, A. Reinforcement Learning, second edition: An Introduction. MIT Press, 2018. (Adaptive Computation and Machine Learning series). ISBN 9780262039246. Disponível em: <<https://books.google.com.br/books?id=sWVODwAAQBAJ>>.
- XU, J. et al. Imagereward: Learning and evaluating human preferences for text-to-image generation. Advances in Neural Information Processing Systems, v. 36, p. 15903–15935, 2023.
- BLACK, K. et al. Training diffusion models with reinforcement learning. arXiv preprint arXiv:2305.13301, 2023.
- FAN, Y. et al. Dpok: Reinforcement learning for fine-tuning text-to-image diffusion models. Advances in Neural Information Processing Systems, v. 36, p. 79858–79885, 2023.
- MIAO, Z. et al. Training diffusion models towards diverse image generation with reinforcement learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. [S.l.: s.n.], 2024. p. 10844–10853.
- ZHANG, Y. et al. Large-scale reinforcement learning for diffusion models. In: SPRINGER.
- European Conference on Computer Vision. [S.l.], 2024. p. 1–17.
- CHOI, J. et al. Perception prioritized training of diffusion models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. [S.l.: s.n.], 2022. p. 11472–11481.
- KIM, M. et al. Adaptive non-uniform timestep sampling for diffusion model training. arXiv preprint arXiv:2411.09998, 2024.
- WANG, K. et al. A closer look at time steps is worthy of triple speed-up for diffusion model training. In: Proceedings of the Computer Vision and Pattern Recognition Conference. [S.l.: s.n.], 2025. p. 12934–12944.